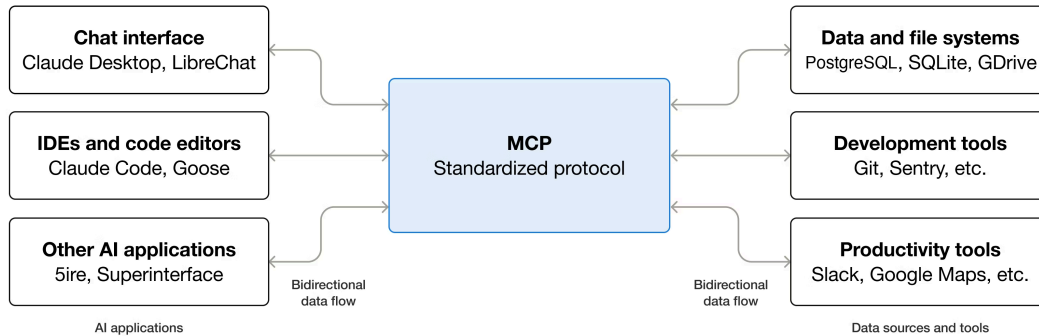


Week 11. MCP

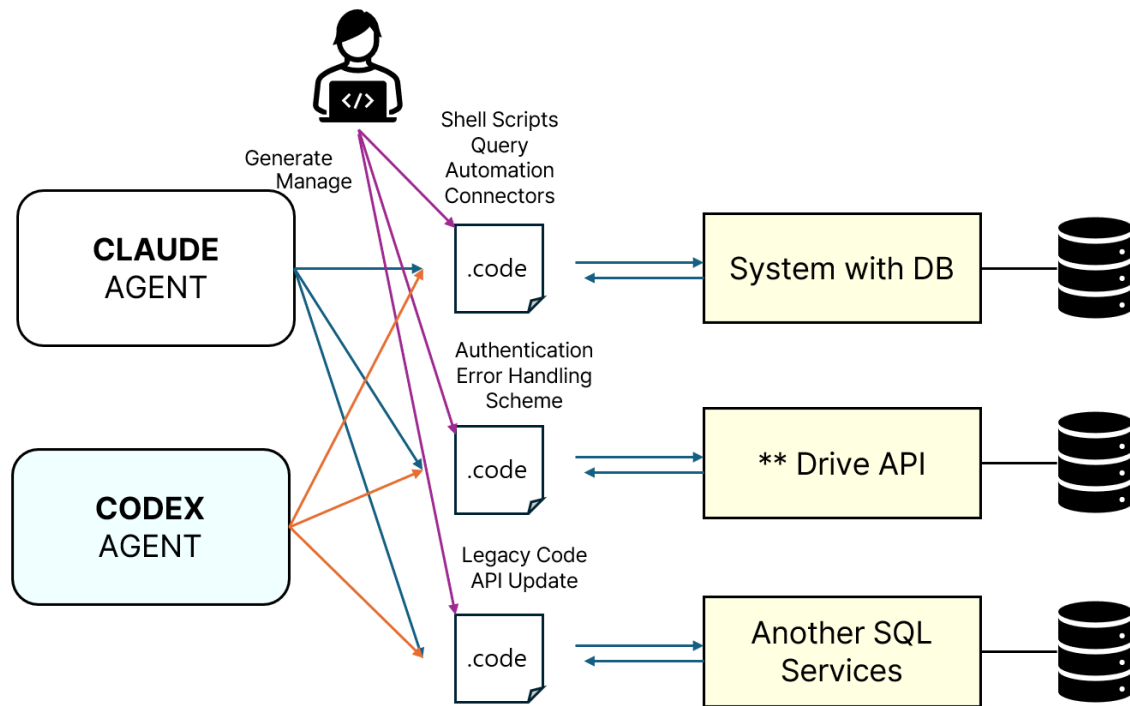
Model Context Protocol



Anthropic이 만들고, Open AI, Google 등이 참여하며 표준화된 프로토콜

MCP는 Claude, Codex, Gemini 같은 AI 에이전트가 다양한 외부 시스템과 동일한 방식으로 연결될 수 있도록 만드는 "AI용 USB 인터페이스"에 가까움

Why MCP

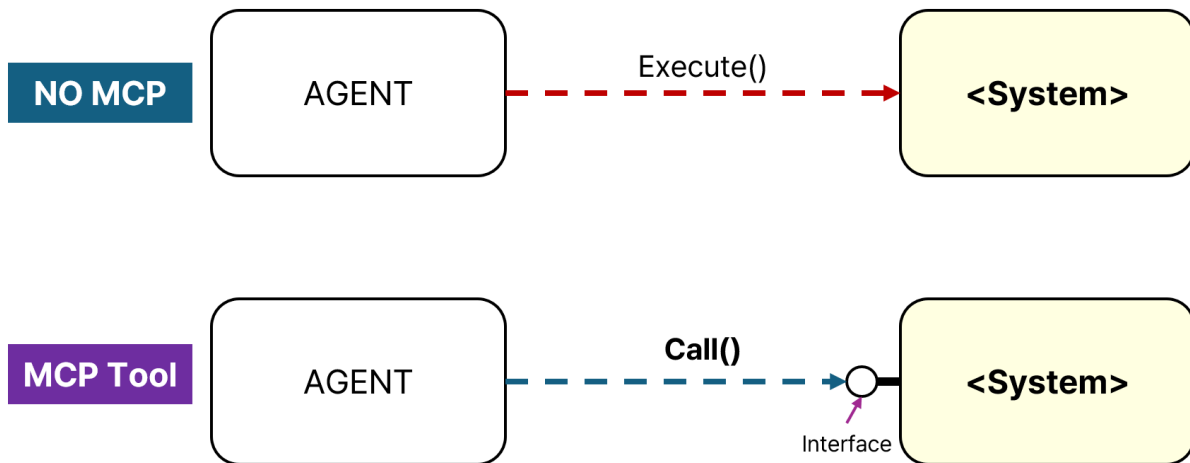


- AI 에이전트가 DB, System, API 등을 이해 하려면 이를 분석할 수 있는 코드를 작성하여 긴 시간동안 분석, 커넥터 코드를 작성하거나 사람이 도구를 직접 만들어야 했음 (~2025)
- 당시에 Slack, Drives, Github 등 인간 개발자 혹은 AI Agent가 접근 할 수 있는 API는 개별 서비스 제공자나 시스템에 엔지니어가 쉽게 적용 가능했고, AI Agent는 해당 API를 핸들링 할 수 있는 코드/문서를 분석하여 사용하면 OK 였음
- 모델 성능, 비용, 기억력(Storage/MEMORY), 병렬 에이전트 관리, 하네스 등이 어우러진 지금 시대에 와서는 확 와닫지 않겠지만 이 당시에는 성능이 좋지 않아 매번 복잡한 분석 작업이 반복되고, 하나의 서비스와 연결된 LLM 에이전트 시스템을 구축하고 검증는 것은 꽤 어려운 작업에 해당

→ 이를 하나의 표준화된 프로토콜을 사용하게 되면, 각 시스템의 API를 에이전트와 연동하는 비용이 줄고, 성능 또한 개선됨

결국엔 OOM이다:

- **캡슐화, 추상화, 다형성** : MCP를 사용하면 Tool Calling으로 일어나는 내부의 일에 대해 AI Agent가 신경 쓸 필요가 없음.



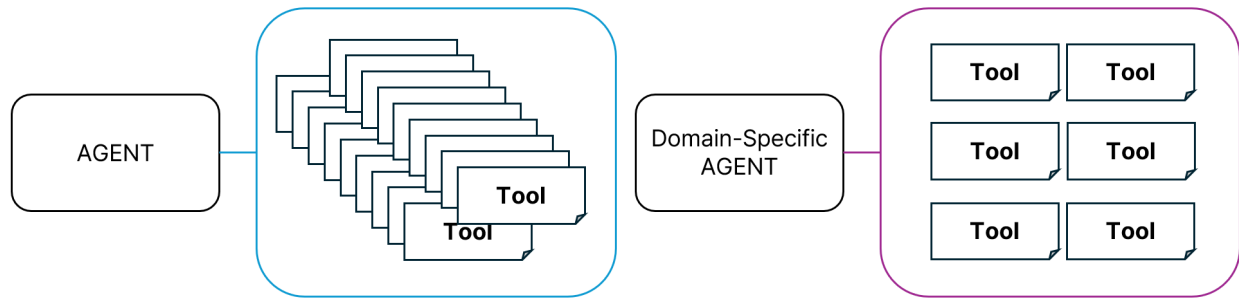
Tradeoff of MCP



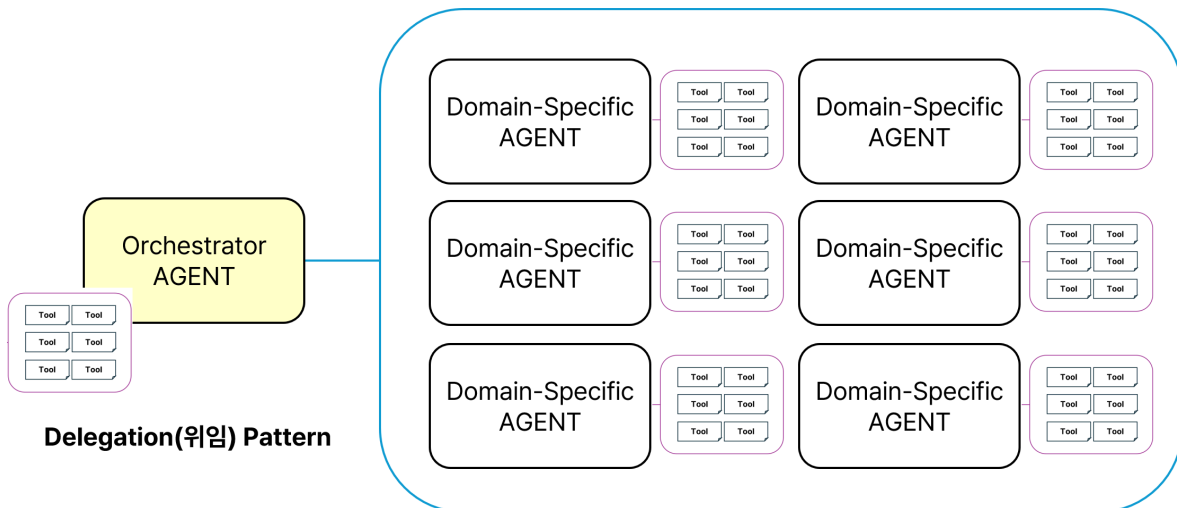
이 맥가이버 나이프 안에서 박혀있는 못을 뽑아낼 수 있는 도구 하나를 찾으려면 ?

→ 각 도구에 대한 기능 추론 → **Context Explosion**

Subagents



Orchestrator-Worker



Continue..

실습 1



이미지 내 내용은 샘플입니다.

Step 1

<https://github.com/INHA-SELAB/AgentMEMO>

- Clone AgentMEMO
- Clone한 디렉토리에서 AI에게 프로젝트의 목표 분석 지시

Step 2

- 자신의 AI 에이전트와 논의하여 입력 스키마를 결정합니다.
- FastMCP를 이용하여 agentmemo.server 의 tools 구현

Step 3

AI 에이전트와 함께 다음 Task를 순차적으로 검증합니다.

1. agentmemo-server가 127.0.0.1:8000에서 실행
2. /mcp POST 수신
3. tools/list에 대해 memo.create, memo.get, memo.list, memo.update, memo.append 반환
4. tools/call로 memo.create 호출 검증
5. SQLite에 저장됐는지 MemoRepository 또는 GUI에서 확인